Intermediate (chunking method changes the answer) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper Python, including how a real chunker handles text that doesn't cooperate). Read Basic first even if you already know Python — it's the fastest way to spot exactly what Intermediate adds.

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and almost the whole thing

Back to the exercise · Hint 1 · Hint 2 · Solution

Hint 1 — The idea, and the exact pieces

Basic Version

There isn't one correct way to split a document into pieces. You're going to try two different ways of splitting the *same* document, then ask the *same* question against both, and see whether the answer you actually get back changes.

One way: cut the text into fixed-size pieces, just counting characters, without caring what word or sentence you happen to be in the middle of.

Another way: cut the text wherever the document already has a natural paragraph break.

Before writing any code, write down a guess: which of these two feels more likely to cut a fact in half by accident? You'll check whether you're right in a minute.

Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

Same idea, as two functions: `chunk_by_chars()` and `chunk_by_paragraph()`, doing two different jobs on the same text.

`chunk_by_chars` treats the whole document as one long string and cuts it every `N` characters, with no idea where a sentence or even a word ends — it can slice straight through the middle of a word.

`chunk_by_paragraph` treats a blank line as the natural place to cut, because that's where the person who wrote the document already grouped their own ideas.

The exact pieces:

- Fixed-size slicing: `for i in range(0, len(text), chunk_size): chunks.append(text[i:i+chunk_size])`.
- Paragraph splitting: `text.split("\n\n")`, then `.strip()` each piece and drop any that are empty.
- Reuse `get_embedding` and `cosine_similarity` from `embedding_similarity` unchanged — you're applying the same math to more chunks, not learning new math.

Both `chunk_by_chars` and `chunk_by_paragraph`, as written so far, can fail in ways that only show up on a document that isn't as tidy as your test document.

`chunk_by_paragraph` assumes the document actually *has* blank-line breaks. Hand it one giant, single-paragraph wall of text — a scraped web page, a PDF that lost its formatting on the way in — and `text.split("\n\n")` returns exactly 1 "chunk": the entire document, unsplit. That's not a crash, which is what makes it dangerous — it silently defeats the entire point of chunking, and you won't notice until a search for something near the end of that giant chunk comes back oddly.

`chunk_by_chars` has a different, subtler problem: even where it "works," any fact sitting near a `chunk_size` boundary gets cut clean in half between two *non-overlapping* chunks, and neither chunk alone contains the whole fact — this is exactly the failure the `chunk_boundary_split` exercise builds on purpose. The real fix isn't a bigger `chunk_size` (that just moves the problem, it doesn't remove it) — it's giving neighboring chunks a little **overlap**, so text near a boundary shows up whole in at least one chunk instead of split across two.

The extra pieces:

- A `chunk_by_chars` that steps forward by `chunk_size - overlap` instead of `chunk_size`, so each new chunk starts a little *before* where the last one ended.

- A fallback inside `chunk_by_paragraph`: if splitting on `"\n\n"` produces only 1 piece (or the piece is still enormous), fall back to `chunk_by_chars` instead of silently returning one giant "chunk."
- The chunker as a parameter: a function is a value, so `best_chunk_for(chunk_fn, text, question_vector)` can call whichever chunker it's given. Its type hint is `Callable[[str], list[str]]` (from `typing`) — "takes a `str`, returns a `list[str]`". Keep the chunkers in a `dict` of name → function and loop over it.

Sketch the overlapping version of `chunk_by_chars` yourself before checking Hint 2.

Difference between Basic and Intermediate: Basic assumes the test document behaves — tidy paragraphs, and no fact unlucky enough to sit exactly on a fixed-size boundary. Intermediate also asks what happens when a document doesn't cooperate: a wall of text with no paragraph breaks at all, or a fact that lands right on a cut point. The fixes — overlap, and a fallback when paragraph-splitting finds nothing to split on — are exactly the kind of thing that separates a chunker that works on your one test file from one that survives a folder of real, messy documents.

Back to the exercise · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and almost the whole thing

Basic Version

```
write a test document: a few made-up paragraphs, all about one topic

make chunk_by_chars(text, chunk_size):
    cut text into pieces of chunk_size characters each, one after another

make chunk_by_paragraph(text):
    split text wherever there's a blank line
    remove any empty pieces

run both functions on the same test document
    -> now you have two different lists of chunks for the same text

pick one question whose answer lives inside one specific paragraph

embed the question, and embed every chunk from both methods

for each method:
    compare the question to every chunk in that method's list
    find the chunk with the highest similarity score

print the top chunk (and its score) for each method

compare: does one method's top chunk contain the whole answer,
         while the other method's top chunk is cut off mid-sentence?
```

Here's almost the whole thing — try running it and reading it line by line:

```
# chunking_practice.py — Intermediate section
import math
from openai import OpenAI

client = OpenAI()

def chunk_by_chars(text, chunk_size=200):
    chunks = []
    for i in range(0, len(text), chunk_size):
```

```
        chunks.append(text[i:i+chunk_size])
    return chunks

def chunk_by_paragraph(text):
    chunks = []
    for piece in text.split("\n\n"):
        stripped = piece.strip()
        if stripped:
            chunks.append(stripped)
    return chunks

def get_embedding(text):
    response = client.embeddings.create(
        model="text-embedding-3-small", input=text
    )
    return response.data[0].embedding

def cosine_similarity(a, b):
    dot = 0.0
    for x, y in zip(a, b):
        dot = dot + x * y
    squares = 0.0
    for x in a:
        squares = squares + x * x
    norm_a = math.sqrt(squares)
    squares = 0.0
    for x in b:
        squares = squares + x * x
    norm_b = math.sqrt(squares)
    return dot / (norm_a * norm_b)
```

Expected output if you run just this: nothing yet — add your test document, question, and the comparison loop below it.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

```
define:
    DOCUMENT = "... a few paragraphs, one topic ..."
    QUESTION = "... a question whose answer lives in one paragraph ..."

define:
    def find_best_chunk(chunks, question_vector) -> tuple[str, float]:
        embed every chunk, score it against question_vector, keep the best

main:
    char_chunks = chunk_by_chars(DOCUMENT)
    paragraph_chunks = chunk_by_paragraph(DOCUMENT)
    question_vector = get_embedding(QUESTION)

    for each method's chunk list:
        best_chunk, best_score = find_best_chunk(chunks, question_vector)

    print both methods' top chunk and score, side by side
    compare which one actually contains the full answer
```

```
# chunking_practice.py — Intermediate section
def find_best_chunk(
    chunks: list[str], question_vector: list[float],
) -> tuple[str, float]:
    best_chunk = ""
    best_score = -1.0
    for chunk in chunks:
        chunk_vector = get_embedding(chunk)
        score = cosine_similarity(question_vector, chunk_vector)
        if score > best_score:
            best_score = score
            best_chunk = chunk
    return best_chunk, best_score
```

Write `DOCUMENT`, `QUESTION`, and the `main()` that calls everything, then compare both against the Solution.

Once that's working, add the overlap and fallback pieces:

```
function chunk_by_chars_with_overlap(text, chunk_size, overlap):
    chunks = []
    i = 0
    while i < len(text):
        chunks.append(text[i : i + chunk_size])
        i = i + (chunk_size - overlap)     # step forward LESS than a chunk_size
    return chunks

function chunk_by_paragraph_safe(text):
    pieces = chunk_by_paragraph(text)
    # blank-line split found nothing useful
    if len(pieces) <= 1:
        return chunk_by_chars_with_overlap(text, chunk_size=200, overlap=40)
    return pieces
```

Here's almost the whole thing — fill in the missing piece yourself:

```
# chunking_practice.py — Intermediate section
def chunk_by_chars_with_overlap(
    text: str, chunk_size: int = 200, overlap: int = 40,
) -> list[str]:
    chunks: list[str] = []
    i = 0
    step = chunk_size - overlap
    while i < len(text):
        chunks.append(text[i:i + chunk_size])
        # your turn: move i forward by `step`, not by `chunk_size` --
        # that's what makes each new chunk start inside the last one
        ...
    return chunks


def chunk_by_paragraph_safe(text: str) -> list[str]:
    pieces = chunk_by_paragraph(text)
    # your turn: if pieces has 1 item or fewer, the document had no real
    # paragraph breaks -- fall back to chunk_by_chars_with_overlap instead
    ...
```

Then pass the chunker in as a value:

```
from typing import Callable


def best_chunk_for(
    chunk_fn: Callable[[str], list[str]],
```

```
    text: str,
    question_vector: list[float],
) -> tuple[str, float]:
    # your turn: call chunk_fn on text, then reuse find_best_chunk
    ...
```

Fill in the overlap step, the fallback check and `best_chunk_for` yourself, then compare all of your finished versions against the Solution.

Difference between Basic and Intermediate: same two chunking ideas (fixed-size, paragraph-based) at 2 completeness levels. Basic produces non-overlapping chunks and trusts that the document actually has paragraph breaks to split on. Intermediate adds a `step` smaller than `chunk_size` so neighboring chunks share text at their edges, and a fallback that stops `chunk_by_paragraph` from silently returning "1 giant chunk" on a document with no blank lines at all — both of which only matter once you stop testing against the one tidy document you wrote for this exercise.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Full solution: Show me the solution